

GLOBAL SOLUTION 2026/1 · O ESPAÇO É A NOVA FRONTEIRA



OrbitCast

API RESTful para planejamento e simulação de transmissões via satélite em regiões remotas ou de baixa conectividade.

DISCIPLINA

Domain Driven Design Using Java

INSTITUIÇÃO

FIAP

TURMA

1TDSPB

SOLUÇÃO

OrbitCast

EQUIPE — COD3X

Gabriel Stuani — RM 566682 · Guilherme Soares — RM 568227 · Erick Ramos — RM 567837 ·
Matheus Carneiro Maciel — RM 567753

Sumário

1. Objetivo e Escopo do Projeto	3
2. Descrição da Solução	4
3. Arquitetura e Estrutura do Projeto	5
4. Modelo de Dados (Banco de Dados)	6
5. Principais Funcionalidades e Métodos	7
6. Tabela de Endpoints (API REST)	10
7. Tratamento de Exceções e Padrão de Erro	11
8. Protótipo — Telas do Sistema	12
9. Instruções de Execução	16
10. Links de Entrega	17

Documento técnico do projeto OrbitCast — disciplina Domain Driven Design Using Java (FIAP, Global Solution 2026/1).

1. Objetivo e Escopo do Projeto

Objetivo

O **OrbitCast** é uma plataforma de **simulação e apoio à decisão** para o planejamento de transmissões via satélite, voltada a canais de TV, transmissoras, produtoras, instituições educacionais, órgãos públicos e organizadores de eventos que precisam levar conteúdo a **regiões remotas ou de baixa conectividade**.

A proposta **não** opera satélites reais nem realiza transmissões físicas: o sistema centraliza cadastros e calcula **estimativas de custo, alcance, qualidade de sinal e viabilidade**, gerando recomendações de estratégia de cobertura.

Conexão com o tema da Global Solution

O projeto aplica o conceito de **infraestrutura espacial a problemas reais da Terra** — conectividade, comunicação e novos modelos de negócio — aderente ao tema “O espaço é a nova fronteira”.

Escopo entregue

- API RESTful completa (Quarkus/Java 21) com camadas de Domain Driven Design.
- CRUD de clientes, canais, regiões, campanhas e planos de cobertura.
- Motor de **simulação** de custo/alcance/qualidade/viabilidade e **dashboard** com métricas agregadas.
- Persistência em banco relacional (H2 local e Oracle em produção), deploy em servidor (Render) e integração com Front-End.

Status de implementação: a solução foi **completamente implementada** no backend (API + banco + deploy) e integrada ao Front-End da disciplina, conforme demonstrado nas seções 5 e 8.

2. Descrição da Solução

Problema abordado

Muitas regiões ainda têm baixa qualidade de internet ou pouca estrutura de comunicação. Ao mesmo tempo, empresas, escolas, canais regionais e órgãos públicos podem precisar transmitir conteúdo para essas áreas. Hoje não existe uma forma simples de **simular**:

- quanto custaria transmitir para determinada região;
- qual área teria maior alcance e qual região teria maior necessidade;
- qual qualidade de sinal seria viável e se o orçamento informado é suficiente;
- qual plano de cobertura faz mais sentido.

Como o OrbitCast resolve

O usuário cadastra **clientes** e seus **canais**, registra **regiões** de interesse (com população, índice de conectividade, área e prioridade social) e cria **campanhas de transmissão** informando período, duração, qualidade desejada e orçamento. Ao associar regiões e executar a **simulação**, o sistema calcula custo estimado, alcance, qualidade de sinal e classifica a viabilidade em **BAIXA / MÉDIA / ALTA**, com uma recomendação textual de estratégia.

Exemplo de uso

Uma instituição quer transmitir aulas para cidades de baixa conectividade. Ela cadastra o canal educacional, seleciona regiões do Norte e Centro-Oeste, informa o orçamento e o sistema calcula o custo estimado, o público alcançado e classifica a viabilidade — apoiando a decisão antes de qualquer investimento.

3. Arquitetura e Estrutura do Projeto

O OrbitCast segue o padrão **Domain Driven Design** em camadas, com separação clara de responsabilidades. A requisição HTTP entra pela camada **Resource** (JAX-RS), que delega regras de negócio à camada **BO**, que por sua vez usa os **DAO** (JDBC manual) para acessar o banco através da **Connection**.



Organização de pastas

```
src/main/java/br/com/fiap/orbitcast
├─ entities/      classes-modelo (Cliente, Canal, Regiao, Campanha...)
├─ connection/   DatabaseConnection, DatabaseInitializer
├─ dao/          CRUD JDBC por entidade
├─ bo/           regras de negócio + ValidationUtils
├─ dto/          DashboardResumo
├─ exceptions/   exceções + ApiExceptionMapper + ApiError
├─ resources/    endpoints REST (JAX-RS)
src/main/resources/db      scripts H2 (schema.sql, data.sql)
src/main/resources/db/oracle scripts Oracle (schema.sql, data.sql, drop.sql)
```

4. Modelo de Dados (Banco de Dados)

O banco possui **7 tabelas** com relacionamentos 1:N e um relacionamento **N:N** resolvido pela tabela associativa `T_OC_CAMPANHA_REGIAO`. As tabelas seguem o padrão Oracle/Java definido para o projeto: nomes de tabela em maiúsculas com prefixo `T_OC_`, atributos em minúsculas e sufixo `_id` apenas para identificadores técnicos e chaves estrangeiras. As tabelas usam chaves primárias autoincrementadas, chaves estrangeiras, e restrições `NOT NULL`, `UNIQUE` e `CHECK`.

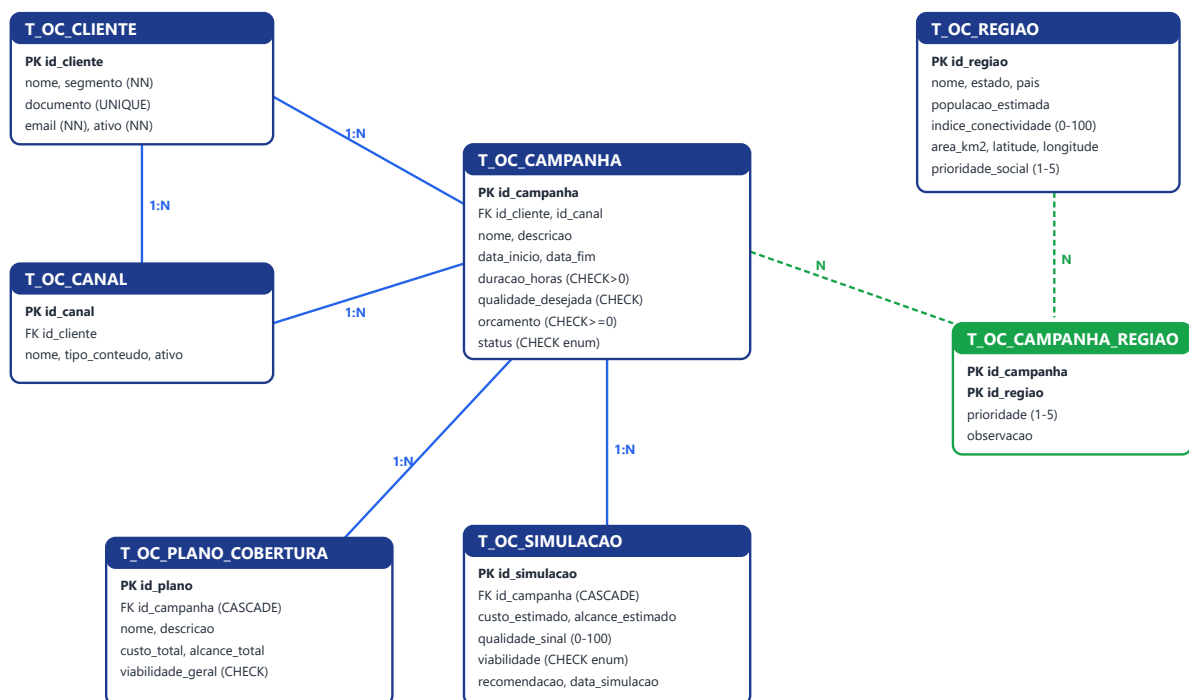


Figura 1 — Diagrama Entidade-Relacionamento (7 tabelas). Linhas tracejadas verdes = relacionamento N:N via tabela associativa `T_OC_CAMPANHA_REGIAO`.

Tabelas e relacionamentos

Tabela	Papel	Relacionamentos
T_OC_CLIENTE	Contratante das campanhas	1:N com canais e campanhas
T_OC_CANAL	Canal/transmissora do cliente	N:1 cliente · 1:N campanhas
T_OC_REGIAO	Área-alvo de cobertura	N:N com campanhas
T_OC_CAMPANHA	Campanha de transmissão	N:1 cliente/canal · 1:N simulações/planos · N:N regiões
T_OC_CAMPANHA_REGIAO	Tabela associativa (N:N) — PK composta	liga campanhas ↔ regiões (prioridade, observação)
T_OC_SIMULACAO	Resultado calculado da simulação	N:1 campanha (ON DELETE CASCADE)
T_OC_PLANO_COBERTURA	Plano de cobertura da campanha	N:1 campanha (ON DELETE CASCADE)

Integridade: PKs autoincrementadas; FKs em todas as ligações; **UNIQUE** em `T_OC_CLIENTE.documento`; **CHECK** em faixas (conectividade 0–100, prioridade 1–5, duração > 0, orçamento ≥ 0) e em enums (qualidade, status, viabilidade); **ON DELETE CASCADE** de campanha para simulações, planos e associações de região.

5. Principais Funcionalidades e Métodos

As regras de negócio ficam na camada **BO**. Abaixo, as principais funcionalidades com a explicação dos métodos e prints reais do sistema em execução.

5.1 Validações de negócio (ValidationUtils + BOs)

- **Documento (CPF/CNPJ):** `optionalCpfCnpj()` valida os **dígitos verificadores** reais — aceita 11 dígitos (CPF) ou 14 (CNPJ) e rejeita sequências repetidas.
- **E-mail:** `requireEmail()` valida formato por expressão regular; **telefone:** `optionalPhone()` exige DDD válido e 10/11 dígitos.
- **Texto e faixas:** `requireText()`, `requireBetween()` garantem obrigatoriedade, limites de tamanho e intervalos (ex.: prioridade 1–5).

5.2 CRUD de Campanhas (CampanhaTransmissaoBo)

- `cadastrar()` / `atualizar()`: validam que o **cliente e o canal existem e estão ativos**, que o **canal pertence ao cliente**, que **data_início ≤ data_fim**, que a **duração não excede o período** e que **orçamento ≥ 0**. Campanha **APROVADA** exige ao menos uma região associada.
- Máquina de estados: campanha **CANCELADA** ou **FINALIZADA** **não pode ser alterada** nem simulada.
- `adicionarRegiao()` / `removerRegiao()`: gerenciam o relacionamento **N:N**, impedindo associação duplicada e validando prioridade (1–5).

The screenshot shows the OrbitCast web application interface. On the left is a sidebar with navigation links: Dashboard, Campanhas (selected), Regiões, Simulação, Sobre, FAQ, Integrantes, and Contato. Below these is a footer with 'Sistema Operacional v1.0.0 - OrbitCast'. The main content area is titled 'Campanhas de Transmissão' and contains a 'Nova Campanha' form. The form fields are: 'NOME DA CAMPANHA *' with a text input containing 'Ex: Aulas via Satélite'; 'CLIENTE *' with a dropdown menu showing 'Selecione o cliente'; 'CANAL' with a dropdown menu showing 'Selecione o canal'; 'QUALIDADE DESEJADA' with a dropdown menu showing 'HD - High Definition'; 'DATA INICIO' and 'DATA FIM' with date pickers showing 'dd/mm/aaaa'; 'DURAÇÃO (HORAS)' with a text input showing '1'; 'ORÇAMENTO (R\$)' with a text input showing '0'; 'STATUS' with a dropdown menu showing 'Planejada'; and 'DESCRIÇÃO' with a text area containing 'Descreva o objetivo da campanha...'. At the bottom of the form are 'Cancelar' and 'Criar Campanha' buttons.

Figura 2 — Método `cadastrar()` : formulário de criação de campanha (cliente, canal, qualidade, período, duração, orçamento, status).

5.3 Motor de Simulação (SimulacaoBo.simular)

O método `simular(campanhaId)` é o coração da solução. A partir das regiões associadas, calcula:

- **Custo estimado** — `calcularCusto()` : custo/hora por qualidade (SD R\$250 · HD R\$450 · FULL_HD R\$700 · 4K R\$1200) × duração, somado a custos por área, por nº de regiões e por prioridade social.
- **Qualidade de sinal** — `calcularQualidadeSinal()` : base por qualidade, com penalidade por área e bônus por prioridade (limitada entre 45% e 98%).
- **Alcance** — `calcularAlcance()` : população total ponderada por prioridade e por baixa conectividade.
- **Viabilidade** — `classificarViabilidade()` : cruza cobertura do orçamento vs. custo e qualidade, classificando em **ALTA / MÉDIA / BAIXA**, com recomendação textual gerada por `gerarRecomendacao()` .

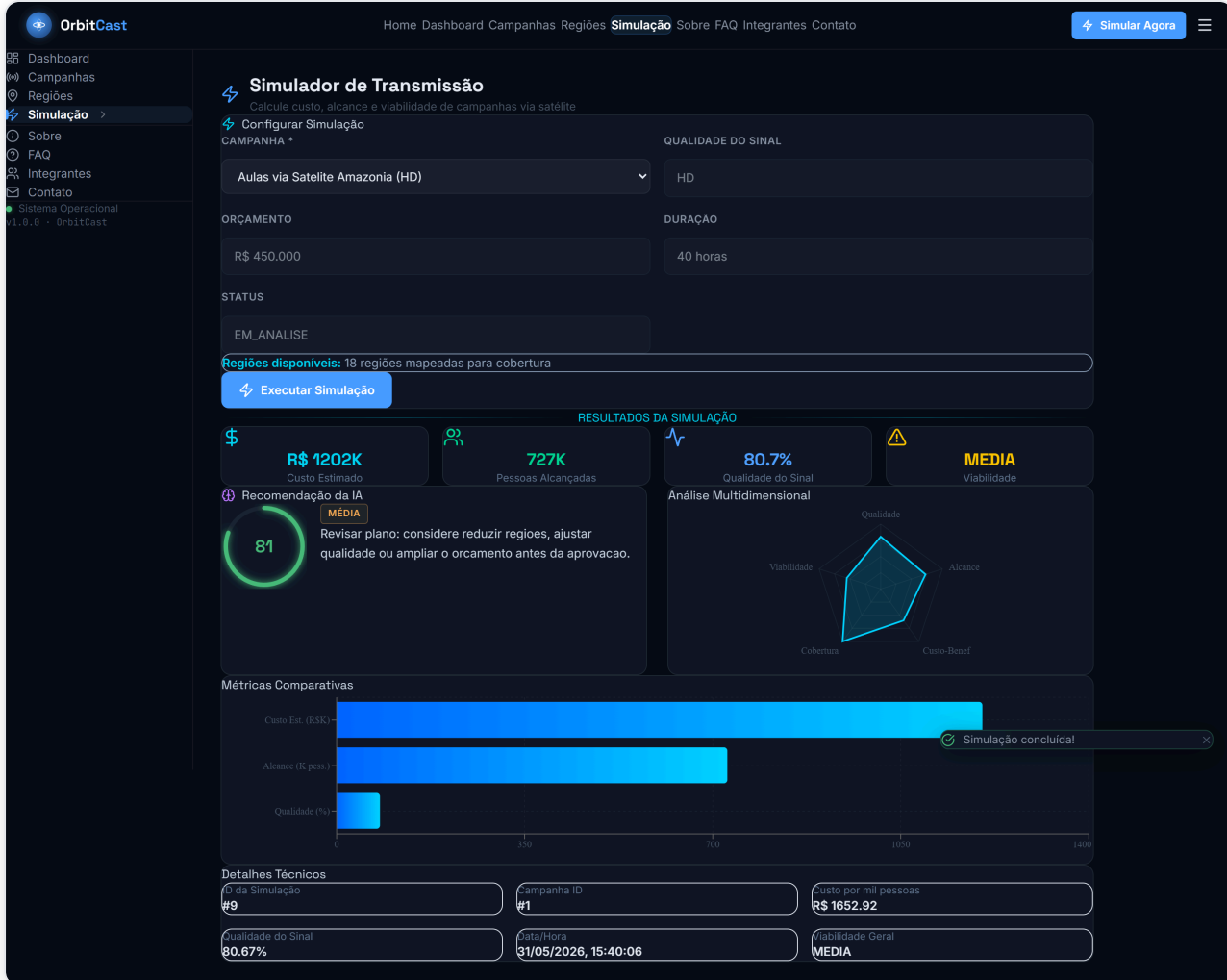


Figura 3 — Método `simular()` em execução: custo, alcance, qualidade, viabilidade e recomendação calculados em tempo real (endpoint `POST /campanhas/{id}/simulacoes`).

5.4 Dashboard (DashboardBo.resumo)

O método `resumo()` agrega métricas (totais, alcance, custo médio, qualidade média) e distribuições (campanhas por status, simulações por viabilidade) para o painel inicial.



Figura 4 — `GET /dashboard/resumo` : métricas e gráficos agregados a partir do banco.

5.5 Detalhe da Campanha (navegação N:N + histórico)

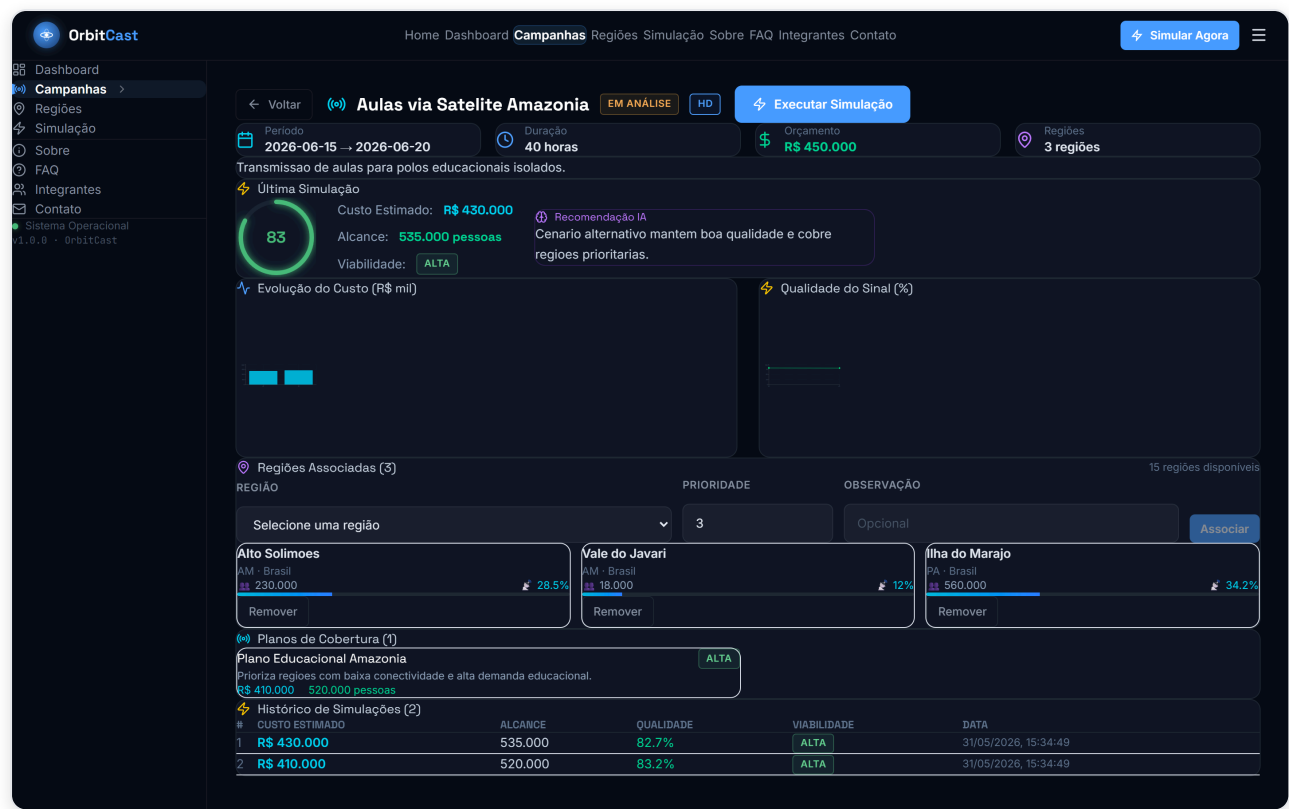


Figura 5 — Tela de detalhe: regiões associadas (N:N), planos de cobertura e histórico de simulações de uma campanha.

6. Tabela de Endpoints (API REST)

Base de produção: `https://orbitcast-global-api.onrender.com` · Formato: `application/json` (exceto `/health`, que retorna `text/plain`). A API segue os princípios REST: recursos no plural, verbos HTTP e códigos de status adequados.

Verbo	URI	Descrição	Status
GET	/health	Verifica se a API está online	200
GET	/dashboard/resumo	Métricas agregadas do sistema	200
Clientes · Canais · Regiões · Planos de Cobertura (CRUD idêntico)			
GET	/clientes · /canais · /regioes · /planos-cobertura	Lista todos os registros	200
GET	/{\$recurso}/{\$id}	Busca um registro por id	200 / 404
POST	/{\$recurso}	Cria um novo registro	201 / 400
PUT	/{\$recurso}/{\$id}	Atualiza um registro existente	200 / 400 / 404
DELETE	/{\$recurso}/{\$id}	Remove um registro	204 / 404

Verbo	URI	Descrição	Status
Campanhas e sub-recursos			
GET	/campanhas · /campanhas/{id}	Lista / busca campanha	200 / 404
POST PUT DELETE	/campanhas · /campanhas/{id}	Cria / atualiza / remove campanha	201 / 200 / 204
GET	/campanhas/{id}/regioes	Regiões associadas à campanha (N:N)	200
POST DELETE	/campanhas/{id}/regioes/{regiaoId}	Associa / desassocia região	201 / 204
GET	/campanhas/{id}/simulacoes	Histórico de simulações da campanha	200
POST	/campanhas/{id}/simulacoes	Executa nova simulação (sem corpo)	201 / 400
GET	/campanhas/{id}/planos-cobertura	Planos de cobertura da campanha	200
GET	/simulacoes · /simulacoes/{id}	Lista / busca simulações	200 / 404

CORS habilitado para os verbos GET, POST, PUT, DELETE e OPTIONS. Contrato completo com exemplos de JSON disponível em `API_CONTRACT.md` no repositório.

7. Tratamento de Exceções e Padrão de Erro

As exceções de domínio são centralizadas e convertidas em respostas JSON padronizadas pelo `ApiExceptionHandler`, garantindo consistência para o Front-End.

Exceção	Significado	Status HTTP
<code>BusinessException</code>	Regra de negócio violada (validação)	400 Bad Request
<code>EntityNotFoundException</code>	Registro não encontrado	404 Not Found
<code>DataAccessException</code>	Falha de acesso a dados	500 Internal Server Error

Formato padrão de erro (`ApiError`)

```
{
  "status": 400,
  "erro": "Regra de negocio violada",
  "mensagem": "Nome do cliente e obrigatorio.",
  "timestamp": "2026-05-26T15:00:00"
}
```

8. Protótipo — Telas do Sistema

Capturas reais do sistema em funcionamento, com o Front-End (React) integrado à API OrbitCast.

Front-End integrado: <https://orbitcast-front.vercel.app>

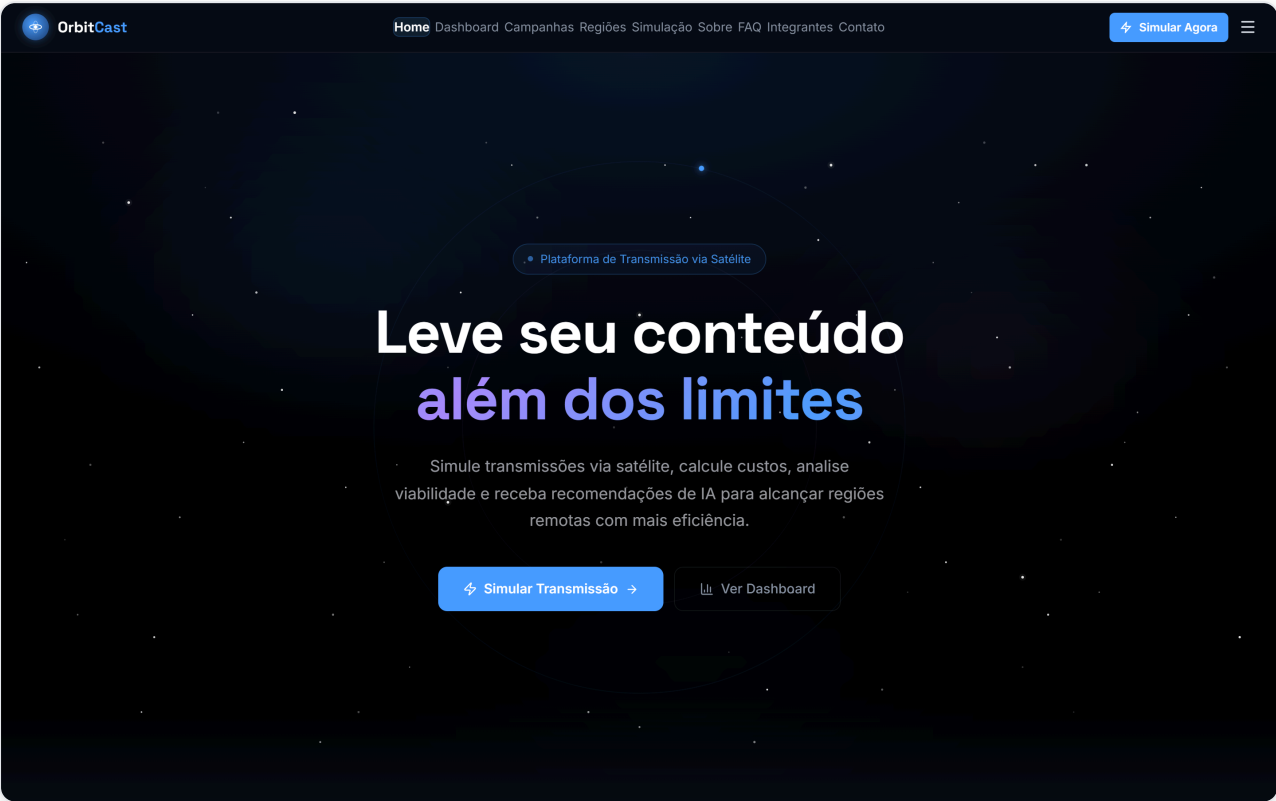


Figura 6 — Página inicial (landing).

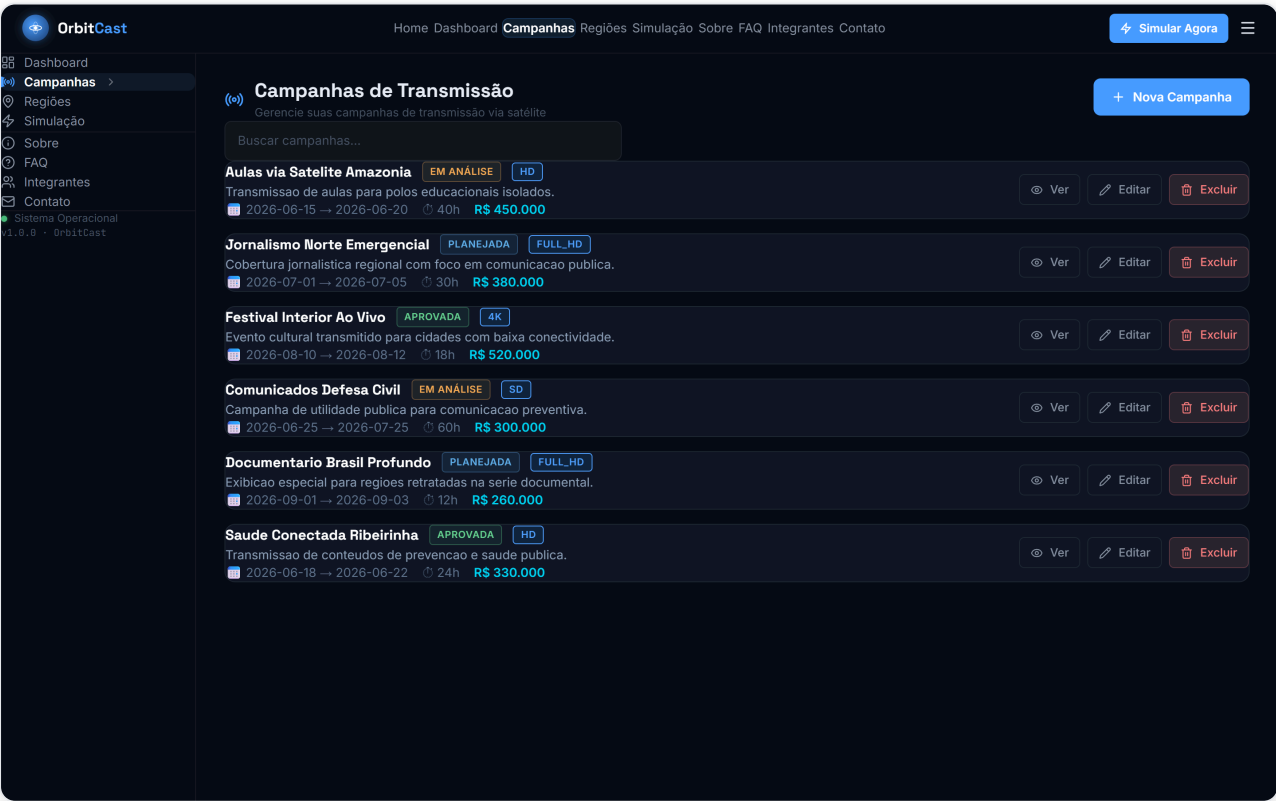


Figura 7 — Listagem de campanhas com status, orçamento e ações de CRUD.

OrbitCast

Dashboard

Campanhas

Regiões

Simulação

Sobre

FAQ

Integrantes

Contato

Sistema Operacional v1.0.0 - OrbitCast

Home Dashboard CampanhasRegiõesSimulaçãoSobreFAQIntegrantesContato

Simular Agora

Regiões de Cobertura

Mapeamento de áreas para transmissão via satélite

Buscar por nome ou estado...

Todos os estados

Qualquer prioridade

18 de 18 regiões

REGIÃO	ESTADO	POPULAÇÃO	CONECTIVIDADE	PRIORIDADE	ÁREA		
Alto Solimões	AM	230.000	29%	CRÍTICA (5)	21.500 km²		
Ilha do Marajo	PA	560.000	34%	CRÍTICA (5)	40.000 km²		
Vale do Javari	AM	18.000	12%	CRÍTICA (5)	85.000 km²		
Sertao Central	CE	410.000	47%	ALTA (4)	18.500 km²		
Pantanal Norte	MT	120.000	38%	ALTA (4)	30.000 km²		
Chapada Diamantina	BA	370.000	52%	MÉDIA (3)	38.000 km²		
Oeste Catarinense Rural	SC	260.000	63%	BAIXA (2)	16.600 km²		
Vale do Ribeira	SP	300.000	59%	MÉDIA (3)	17.000 km²		
Baixo Sao Francisco	AL	210.000	45%	ALTA (4)	9.200 km²		
Fronteira Oeste Gaucha	RS	190.000	62%	BAIXA (2)	29.000 km²		
Medio Xingu	PA	145.000	25%	CRÍTICA (5)	42.000 km²		
Baixada Maranhense	MA	310.000	40%	ALTA (4)	17.000 km²		
Jalapao	TO	36.000	26%	CRÍTICA (5)	34.000 km²		
Pantanal Sul	MS	95.000	37%	ALTA (4)	54.000 km²		
Vale do Jequitinhonha	MG	680.000	49%	ALTA (4)	50.000 km²		
Sertao do Araripe	PE	520.000	43%	ALTA (4)	18.000 km²		
Serra da Capivara	PI	125.000	38%	ALTA (4)	13.000 km²		
Litoral Norte Amapaense	AP	72.000	32%	CRÍTICA (5)	24.000 km²		

Figura 8 — Catálogo de regiões de cobertura (população, conectividade, prioridade, área).

OrbitCast

Home Dashboard CampanhasRegiõesSimulaçãoSobreFAQIntegrantesContato

Simular Agora

Dashboard

Campanhas

Regiões

Simulação

Sobre

FAQ

Integrantes

Contato

Sistema Operacional v1.0.0 - OrbitCast

Simulador de Transmissão

Calcule custo, alcance e viabilidade de campanhas via satélite

Configurar Simulação

CAMPANHA *

Selecione uma campanha

Executar Simulação

Figura 9 — Tela do simulador de transmissão.

OrbitCast

Home Dashboard CampanhasRegiõesSimulação Sobre FAQ Integrantes Contato

Simular Agora

Dashboard

Campanhas

Regiões

Simulação

Sobre

FAQ

Integrantes

Contato

Sistema Operacional v1.0.0 - OrbitCast

Regiões de Cobertura

Mapa de cobertura de áreas para transmissão de sinal

6 Q: 10 por região do estado

18 de 18 regiões

Todos os estados

Qualquer prioridade

REGIÃO	ESTADO	POPULAÇÃO	COBERTURA	PRIORIDADE	ÁREA
Alto Solimões	AM	95.000	100%	1	21.000 km²
Ilha do Marajó	PA	680.000	100%	1	40.000 km²
Vale do Jari	AM	520.000	100%	1	18.000 km²
Sertão Central	PE	125.000	100%	1	15.000 km²
Pantanal Noroeste	MS	72.000	100%	1	24.000 km²
Chapada Diamantina	BA	95.000	100%	1	21.000 km²
Oeste Catarinense	SC	680.000	100%	1	40.000 km²
Vale do Ribeira	MS	520.000	100%	1	18.000 km²
Baixo São Francisco	BA	125.000	100%	1	15.000 km²
Fronteira Oeste	MT	72.000	100%	1	24.000 km²
Médio Xingú	PA	95.000	100%	1	21.000 km²
Baixada Maranhense	MA	680.000	100%	1	40.000 km²
Jalapão	GO	520.000	100%	1	18.000 km²
Pantanal Sul	MS	72.000	100%	1	24.000 km²
Vale do Jequitinhonha	MG	680.000	100%	1	40.000 km²
Sertão do Araripe	PE	520.000	100%	1	18.000 km²
Serra da Capivara	PE	125.000	100%	1	15.000 km²
Litoral Norte Amapaense	AP	72.000	100%	1	24.000 km²

Nova Região

NOME *

ESTADO *

PAÍS

POPULAÇÃO ESTIMADA

ÍNDICE DE CONECTIVIDADE (%)

ÁREA (KM²)

LATITUDE

LONGITUDE

PRIORIDADE SOCIAL (1-5)

Cancelar

Criar Região

Figura 10 — Formulário de cadastro/edição de região.

OrbitCast

Home Dashboard Campanhas Regiões Simulação Sobre FAQ Integrantes Contato

Simular Agora

GLOBAL SOLUTION - FIAP 2025

Equipe OrbitCast

Quatro integrantes da turma 1TDSPB construindo uma experiencia para planejamento de transmissões via satélite, com foco em clareza operacional, dados e integração com API Java.

4integrantes

1TDSPBturma



RM5666821TDSPB

FRONT-END DEVELOPER

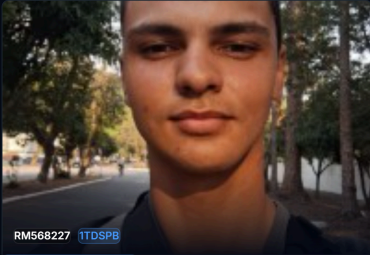
Interface e experiencia

Gabriel Stuani

Atua na composicao visual das telas publicas e na consistencia dos fluxos de navegacao.

</> GitHub

LinkedIn



RM5682271TDSPB

FRONT-END DEVELOPER

Componentes e responsividade

Guilherme Soares

Organiza componentes reutilizaveis e garante que a experiencia funcione bem em diferentes tamanhos de tela.

</> GitHub

LinkedIn



RM5678371TDSPB

FRONT-END DEVELOPER

Integracao e dados

Erick Ramos Santos

Conecta o front-end com a API Java e cuida dos fluxos de dados usados nas simulacoes e dashboards.

</> GitHub

LinkedIn



RM5677531TDSPB

FRONT-END DEVELOPER

Documentacao e qualidade

Matheus Carneiro Maciel

Apoia a organizacao da entrega, validacao das informacoes e acabamento das areas institucionais.

</> GitHub

LinkedIn

Front-End Design Engineering + Domain Driven Design Using Java

Economia Espacial e Conectividade via Satellite

React + ViteTypeScriptTailwind CSSQuarkus

Oracle DBRender

API

Figura 11 — Página de integrantes da equipe.

9. Instruções de Execução

Ferramentas e versões

- **IDE:** IntelliJ IDEA, Eclipse, NetBeans ou VS Code.
- **Java JDK 21** (obrigatório) · **Apache Maven 3.9**.
- **Banco:** H2 (embutido, sem instalação) para execução local; Oracle Database para produção.

Como importar o projeto

1. Clonar o repositório: `git clone https://github.com/Guilherme-Soares00/OrbitCast-Global.git`
2. Abrir a pasta na IDE como projeto **Maven** (o `pom.xml` está na raiz).
3. Aguardar o download das dependências do Maven.

Executar localmente (H2)

```
mvn quarkus:dev
# API disponível em http://localhost:8080
curl http://localhost:8080/health -> "OrbitCast API online"
```

Executar testes

```
mvn test           # testes unitários
mvn verify -DskipITs=false # + testes de integração
```

Executar com Oracle (produção)

```
setx variáveis: ORACLE_JDBC_URL, ORACLE_USERNAME, ORACLE_PASSWORD
mvn quarkus:dev -Dquarkus.profile=oracle
```

Deploy (Render)

Deploy via **Docker** (Web Service no Render, branch `main`, Dockerfile na raiz). Variáveis de ambiente: `QUARKUS_PROFILE=oracle`, `ORACLE_JDBC_URL`, `ORACLE_USERNAME`, `ORACLE_PASSWORD`. A porta é fornecida automaticamente pela plataforma.

Ambiente	URL
Produção	<code>https://orbitcast-global-api.onrender.com</code>
Validação rápida	<code>GET /health</code> · <code>GET /dashboard/resumo</code>

10. Links de Entrega

Item	Link
Repositório (Back-End)	https://github.com/Guilherme-Soares00/OrbitCast-Global.git
Deploy da API	https://orbitcast-global-api.onrender.com
Repositório (Front-End)	https://github.com/FIAP-Cod3X/OrbitCast_front
Deploy do Front-End	https://orbitcast-front.vercel.app
Vídeo Pitch (YouTube)	https://youtu.be/HhciukgoQXE
Vídeo de Demonstração	https://youtu.be/_y9vuN-QIHY